

15.5

CQRS Pattern

Command Query Responsibility Segregation

Java Backend Architecture Interview Series • Tutorial Edition • Easy Diagrams & Examples

The Core Idea

CQRS means splitting your application into two parts: a WRITE side (handles commands that change data) and a READ side (handles queries that return data). Each side is optimized independently.

■ **Simple Analogy:** A library has a 'Returns Desk' (write — you hand in a book) and a 'Catalog Search' (read — you look up books). They are separate — the search system is optimized for fast lookups, the returns desk for accuracy. Combining them would make both worse.

Key Terms

Command	An instruction to change state. Example: PlaceOrderCommand, CancelOrderCommand. May be rejected.
Query	A request for data. Example: GetOrdersQuery. Never changes data.
Command Handler	Processes a command. Validates, updates the write DB, publishes an event.
Query Handler	Reads from the read model. Returns a flat DTO — no business logic.
Projection	A pre-computed, denormalized read model built from events. Optimized for one specific query.
Read Model	A separate data store (e.g., Elasticsearch, Redis) built to serve queries fast.

Diagram — CQRS Flow

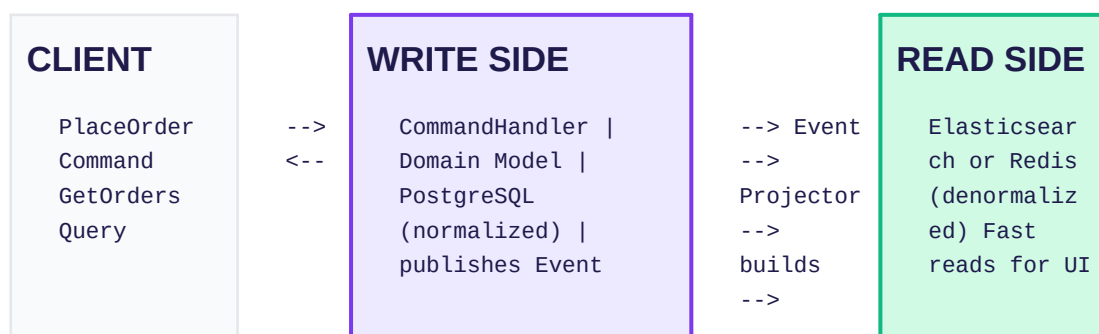


Figure 5: Commands go to write side; queries go to optimized read side

Java Code Example

```
// COMMAND (write side)
@CommandHandler
public void handle(PlaceOrderCommand cmd) {
    Order order = new Order(cmd.getCustomerId(), cmd.getItems());
    order.place(); // domain logic + validation
    orderRepository.save(order);
    eventPublisher.publish(new OrderPlacedEvent(order.getId()));
}

// QUERY (read side – reads from Elasticsearch)
@QueryHandler
public List handle(GetOrdersQuery query) {
    return orderReadRepository.findByCustomerId(query.getCustomerId());
}
```

When to Use CQRS

- Read load is much higher than write load (e.g., 100x more reads)
- Reads need different data shapes than what the write model stores
- You need multiple optimized views of the same data
- Combined with Event Sourcing for full audit history

Caution: CQRS adds complexity. Do NOT use it for simple CRUD applications. The read model is EVENTUALLY CONSISTENT — there can be a short delay (ms to seconds) between a write and the updated read model.

■ **Real-World Example:** Banking: Write side uses PostgreSQL (strict ACID — rejects overdrafts). Read side uses Elasticsearch pre-built from events — handles 50,000 QPS for 'show my transactions' queries without touching the write DB. If read model corrupts, replay all events to rebuild it from scratch.